

# Design Document for **Inventory Manager**

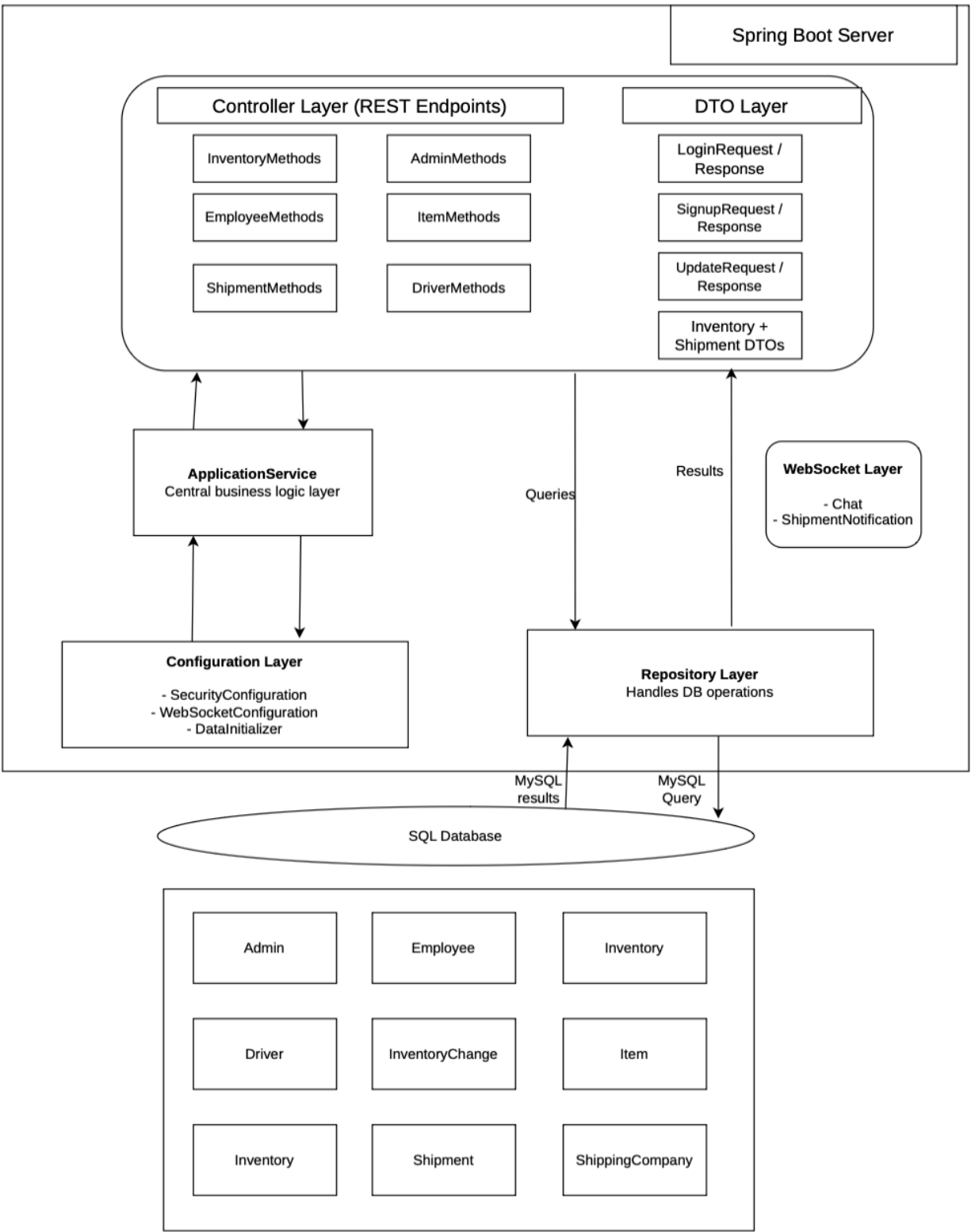
Group **2\_az\_5**

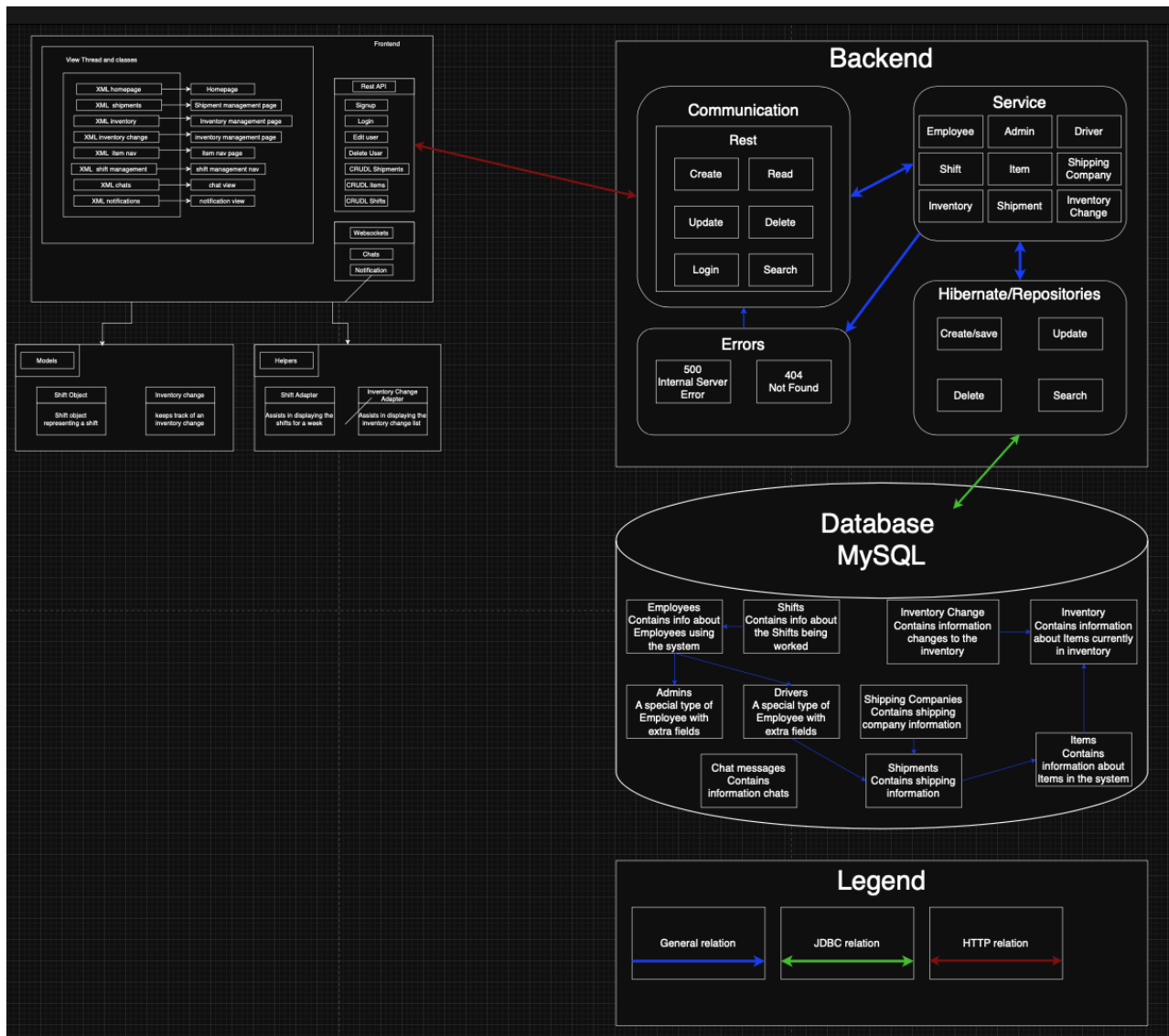
Olivia Blais: 25% contribution

Rishita Kollu: 25% contribution

Ryan Hoefer: 25% contribution

Eli Johnson: 25% contribution





**Frontend:** The user interacts with our frontend application using xml views, each xml view has a corresponding class that handles the logic and sending requests to the server. Some views have more than one request attached to them; for example, our admin shift view has each POST, PUT, GET, and Delete associated with it. For data that requires one parameter, we are using path variables and for multiple parameters, JSON objects are used to help facilitate data transfer. We also have some helper classes and objects to handle displaying lists of JSON objects received from the backend database such as displaying a list of users using an inventory change object that keeps track of the changId, skuNumber, amountOfItem, and employeeId for each change it received from the backend then uses an inventory change adapter to help find which box in the xml to insert the data to.

**Backend:** The backend of the inventory system is made up of a series of modular classes, all representing parts of the system that work together. Each main method class is responsible for controlling HTTP requests and implements its own service class, and each service class implements its own repository. One of the many examples of this being used in our project is with the EmployeeMethods class. EmployeeMethods has all the HTTP endpoints that are used by the frontend, and it handles any logic with the EmployeeService class autowired in. EmployeeService handles logic regarding the Employee objects and implements any logic/search methods that can be used. It also autowires in the EmployeeRepository (an extension of JpaRepository), where the custom search methods are defined.

